

Unit 1: Relational Architecture & Query Optimization

A relational database management system (RDBMS) organizes data into relations (tables) composed of tuples (rows) and attributes (columns) adhering to Codd's 12 relational rules.

Relational algebra provides a formal procedural query language consisting of fundamental operators: Selection (σ) filters rows satisfying a predicate; Projection (π) isolates specific attribute columns; Cartesian Product (cross product) combines tuples from two relations; and Set Difference removes tuples present in a target relation.

Derived operators include Natural Join (join on matching attribute names), Theta Join (join on an arbitrary condition), and Division.

Query processing in an RDBMS transforms a declarative SQL query into an executable physical query plan through four stages: parsing and translation, optimization, code generation, and execution.

The query optimizer evaluates alternative execution plans using cost-based metrics involving disk I/O cost, CPU execution time, and buffer pool usage.

Heuristic query optimization applies semantic transformation rules to relational algebra trees:

1. Pushing selections down as far as possible to reduce intermediate tuple cardinality early.
2. Pushing projections down to eliminate unneeded attribute columns before performing joins.
3. Replacing Cartesian products followed by selection predicates with explicit inner join operators.

Unit 2: Transaction Processing & Serializability

A database transaction is an atomic sequence of database operations executed as a single logical unit of work.

To preserve database integrity, every transaction must adhere strictly to the ACID properties:

1. Atomicity: Requires that all operations of a transaction execute to completion, or none do (all-or-nothing principle).
2. Consistency: Guarantees that a transaction transforms the database from one valid state to another valid state according to all constraints.
3. Isolation: Ensures that concurrently executing transactions execute without interfering with one another, producing results identical to a serial execution.
4. Durability: Ensures that committed transaction modifications persist permanently in non-volatile storage even after hardware or power failures.

Serializability is the fundamental correctness criterion for concurrent transaction execution schedules.

A schedule is conflict serializable if it is conflict equivalent to a serial schedule of the same transactions. Two operations in a schedule conflict if they belong to different transactions, access the same data item, and at least one operation is a write.

Conflict serializability is verified by constructing a Precedence Graph (Serialization Graph) where directed edges ($T_i \rightarrow T_j$) represent conflicting operation precedence. A schedule is conflict serializable if and only if its precedence graph contains no directed cycles.

Unit 3: Concurrency Control Protocols & Two-Phase Locking

Concurrency control protocols guarantee schedule serializability while maximizing transaction throughput and resource utilization in multi-user database systems.

Lock-based protocols use data item locks to restrict concurrent access. Locks exist in two primary modes: Shared locks (S-locks) allow concurrent read access, while Exclusive locks (X-locks) grant exclusive read and write access.

The Two-Phase Locking (2PL) protocol ensures conflict serializability by requiring each transaction to observe two distinct phases:

1. Growing Phase: The transaction acquires required locks but cannot release any locks.
2. Shrinking Phase: The transaction releases acquired locks but cannot request any new locks.

Strict Two-Phase Locking (Strict 2PL) modifies basic 2PL by requiring that all exclusive locks (X-locks) held by a transaction must be retained until the transaction commits or aborts. Strict 2PL eliminates cascading rollbacks (cascading aborts) by preventing other transactions from reading uncommitted modifications.

Rigorous Two-Phase Locking requires that all shared and exclusive locks held by a transaction must be retained until commit time.

Timestamp-based concurrency control assigns a unique monotonically increasing timestamp $TS(T_i)$ to each transaction. The Basic Timestamp Ordering protocol evaluates read and write requests against data item read-timestamps $R-TS(X)$ and write-timestamps $W-TS(X)$ to enforce serializability in timestamp order.

Unit 4: Deadlock Resolution & Recovery Protocols

A deadlock occurs in a lock-based concurrency control system when two or more transactions are in a simultaneous wait state, each waiting for a lock held by another transaction in the cycle.

Deadlock handling strategies fall into two categories:

1. Deadlock Prevention: Protocols prevent deadlocks from occurring by enforcing transaction ordering rules based on timestamps:

- Wait-Die Scheme: Non-preemptive scheme where an older transaction T_i requesting a lock held by a younger transaction T_j is allowed to wait, while a younger transaction requesting a lock held by an older transaction dies (aborts).

- Wound-Wait Scheme: Preemptive scheme where an older transaction T_i wounds (forces abort of) a younger transaction T_j holding the lock, whereas a younger transaction requesting a lock held by an older transaction is allowed to wait.

2. Deadlock Detection & Recovery: The system maintains a Wait-For Graph (WFG) where nodes represent active transactions and directed edges represent lock dependency. A background thread periodically runs cycle-detection algorithms on the WFG. If a cycle is detected, the database selects a victim transaction to abort and roll back to break the cycle.

Database recovery management uses Write-Ahead Logging (WAL) to restore consistency after system crashes. Under WAL, log records recording changes must be flushed to non-volatile disk storage before corresponding dirty database buffer pages are written to data files.

Unit 5: B+ Tree Indexing & Physical File Architecture

Database storage structures organize records into disk blocks to balance storage efficiency and access performance.

Indexing structures accelerate search operations by maintaining auxiliary index files. A primary index is created on a search key that defines the physical ordering of the data file, whereas a secondary index is constructed on non-ordering fields.

A B+ Tree is an N-ary balanced search tree designed for high-fanout disk-based storage:

1. Internal Nodes: Store search key values and child pointers to direct search paths. Internal nodes contain no data records.
2. Leaf Nodes: Store search key values alongside tuple pointers or actual data records. All leaf nodes are doubly linked in sequential key order to support efficient range queries.
3. Node Balance: Every node except the root contains between $\lceil N/2 \rceil$ and N pointers, guaranteeing $\log(N)$ time complexity for search, insertion, and deletion.

During insertion, if a leaf node exceeds capacity N , it splits into two leaf nodes, copying the median search key up to the parent node. During deletion, if a node falls below min capacity, it merges with a sibling or borrows keys through redistribution.